```
// Enhanced Backend: Authentication & Online Booking Integration for Elysian bites
Restaurant
// server/config/mailer.js
const nodemailer = require('nodemailer');

const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST,
  port: process.env.SMTP_PORT,
  auth: {
    user: process.env.SMTP_USER,
    pass: process.env.SMTP_PASS
  }
});

module.exports = transporter;

// server/models/User.js (updated)
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');

const UserSchema = new mongoose.Schema({
  name: String,
  email: { type: String, unique: true, required: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['customer', 'admin'], default: 'customer' },
  emailVerified: { type: Boolean, default: false },
  createdAt: { type: Date, default: Date.now }
});

// Hash password before saving
UserSchema.pre('save', async function(next) {
```

```
 if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

UserSchema.methods.comparePassword = function(candidate) {
  return bcrypt.compare(candidate, this.password);
};

module.exports = mongoose.model('User', UserSchema);

// server/controllers/authController.js
const jwt = require('jsonwebtoken');
const User = require('../models/User');
const transporter = require('../config/mailer');

exports.register = async (req, res) => {
  const { name, email, password } = req.body;
  try {
    const user = new User({ name, email, password });
    await user.save();
    const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '1d' });
    // send verification email
    const url = `${process.env.FRONTEND_URL}/verify/${token}`;
    await transporter.sendMail({
      to: email,
      subject: 'Verify your email',
      html: `Click <a href="${url}">here</a> to verify your email.`
    });
    res.status(201).json({ message: 'Registration successful, please verify your email.' });
  } catch (err) {
```

```
 res.status(400).json({ error: err.message });
  }
};

exports.verifyEmail = async (req, res) => {
  const { token } = req.params;
  try {
    const { id } = jwt.verify(token, process.env.JWT_SECRET);
    await User.findByIdAndUpdate(id, { emailVerified: true });
    res.json({ message: 'Email verified successfully.' });
  } catch (err) {
    res.status(400).json({ error: 'Invalid or expired token.' });
  }
};

exports.login = async (req, res) => {
  const { email, password } = req.body;
  try {
    const user = await User.findOne({ email });
    if (!user) return res.status(401).json({ error: 'Invalid credentials.' });
    const match = await user.comparePassword(password);
    if (!match) return res.status(401).json({ error: 'Invalid credentials.' });
    if (!user.emailVerified) return res.status(401).json({ error: 'Please verify your email first.' });
    const token = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '7d' });
    res.json({ token, user: { id: user._id, name: user.name, email: user.email, role: user.role } });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};

// server/routes/authRoutes.js
```

```
const express = require('express');
const { register, verifyEmail, login } = require('../controllers/authController');
const router = express.Router();

router.post('/register', register);
router.get('/verify/:token', verifyEmail);
router.post('/login', login);

module.exports = router;

// server/models/Table.js
const mongoose = require('mongoose');

const TableSchema = new mongoose.Schema({
  number: Number,
  capacity: Number
});

module.exports = mongoose.model('Table', TableSchema);

// server/controllers/bookingController.js
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const Reservation = require('../models/Reservation');
const Table = require('../models/Table');
const transporter = require('../config/mailer');

// Check availability: returns free tables for given date & time
exports.checkAvailability = async (req, res) => {
  const { date, time, partySize } = req.query;
  const allTables = await Table.find();
```

```
 const reserved = await Reservation.find({ date: new Date(date), time });
  const reservedNumbers = reserved.map(r => r.tableNumber);
  const available = allTables.filter(t => !reservedNumbers.includes(t.number) && t.capacity >= partySize);
  res.json(available);
};


// Create reservation with optional payment
exports.createBooking = async (req, res) => {
  const { customerName, email, phone, date, time, partySize, tableNumber, paymentIntentId } = req.body;
  try {
    // verify table is still free
    const exists = await Reservation.findOne({ date: new Date(date), time, tableNumber });
    if (exists) return res.status(400).json({ error: 'Table already booked.' });


    const booking = new Reservation({ customerName, email, phone, date, time, partySize, tableNumber, paymentIntentId });
    await booking.save();


    // send confirmation email
    await transporter.sendMail({
      to: email,
      subject: 'Your Reservation is Confirmed',
      html: `<p>Dear ${customerName},<br/>Your reservation for ${date} at ${time} (Table ${tableNumber}) is confirmed.<br/>Thank you!</p>`
    });


    res.status(201).json({ message: 'Booking confirmed.', booking });
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
```

```
};

// server/routes/bookingRoutes.js
const express = require('express');
const { checkAvailability, createBooking } = require('../controllers/bookingController');
const router = express.Router();

router.get('/availability', checkAvailability);
router.post('/', createBooking);

module.exports = router;

// server/controllers/orderController.js (update for Stripe & email)
const Order = require('../models/Order');
const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);
const transporterOrder = require('../config/mailer');

exports.createOrder = async (req, res) => {
  try {
    const { customerName, email, items } = req.body;
    const total = items.reduce((sum, i) => sum + i.price * i.quantity, 0) * 100; // in cents

    // create Stripe payment intent
    const paymentIntent = await stripe.paymentIntents.create({
      amount: total,
      currency: 'usd',
      receipt_email: email
    });

    const order = new Order({ customerName, email, items, total: total / 100, paymentIntentId:
paymentIntent.id });
```

```
 await order.save();


   // send order confirmation
   await transporterOrder.sendMail({
     to: email,
     subject: 'Order Confirmation',
     html: `<p>Thank you for your order, ${customerName}.<br/>Your order ID: ${order._id}</p>`
   });


   res.json({ clientSecret: paymentIntent.client_secret });
  } catch (err) {
   res.status(500).json({ error: err.message });
  }
};

// server/server.js (mount new routes)
require('dotenv').config();
const express = require('express');
const connectDB = require('./config/db');

const app = express();
app.use(express.json());

connectDB();

app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/booking', require('./routes/bookingRoutes'));
app.use('/api/menu', require('./routes/menuRoutes'));
app.use('/api/reservations', require('./routes/reservationRoutes'));
app.use('/api/orders', require('./routes/orderRoutes'));
app.use('/api/promotions', require('./routes/promotionRoutes'));
```

```
const PORT = process.env.PORT || 5000;

app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```